

로그 RCA 증거 보존형 KV Cache 압축

# Role D 실전 실행 가이드

리드 / 평가 트랙 — 주차별 행동 · 라이브러리 · 알고리즘 · 판정식

LogPress Project · 16-Week Roadmap

2026. 07



## 0. 한눈에 보기

Role D(리드/평가 트랙)의 무기는 코드가 아니라 **통계**다. 어느 관문에서 무엇을 검정하고, 어떤 수치가 나오면 GO/NO-GO를 외칠지 미리 못박아두는 것이 Role D의 본질이다. 이 문서는 그 판단을 주차별로 실행 가능한 코드 수준까지 내린다.

### 0.1 핵심 3줄 요약

- **Kill-switch 게이트**는 AUC + 순열검정(permutation test)으로, **Tier-1 보존율 비교**는 McNemar 검정(동일 cause 라인에 대한 짝지은 이진 결과)으로, **Tier-2 RCA 정확도 델타**는 McNemar + 부트스트랩 신뢰구간으로 판정한다.
- 4 baseline × 3 strata × 5 압축비 = 최대 60개 검정이므로 다중비교는 **Benjamini-Hochberg(BH) FDR**로 보정하고, Holm으로 강건성을 점검한다.
- 판정식은 **사전에** 확정한다(사후 조정은 p-hacking): GO ①  $n_{\text{hard}} \geq 300$ , ②  $\text{AUC} \geq 0.65$  &  $p < 0.05$  / Tier-1 PASS = hard subset에서 BH 보정 후 유의 우위 / Tier-2 PASS = 델타 > 0 이고 부트스트랩 95% CI 하한 > 0.

### 0.2 판정식 요약표 (Decision Rules)

| 관문                  | 주차      | 판정식 (사전 확정)                                                                                                                              | NO-GO 대응                                                   |
|---------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| ① Kill-switch (M1)  | 1-3주    | ① $n_{\text{hard}} \geq 300$ AND ② 결합신호 $\text{AUC} \geq 0.65$ & 순열검정 $p < 0.05$ (보조) level-filter 보존율 < 0.99                            | 표본 부족 → BGL 병합 / 층화 완화 / 라인 단위 검정. 신호 무력 → 신호 재설계 또는 주제 리셋 |
| Tier-1 평가 (M5, 안전망) | 10-13 주 | hard subset(non-ERROR + middle)에서 LogPress가 4개 baseline 모두 대비 <b>BH 보정 후 <math>p_{\text{bh}} &lt; 0.05</math></b> 로 보존율 우위, 방향 일관(b > c) | overall만 이기면 불충분 → 신호 재설계                                  |
| Tier-2 평가 (M6, 결정타) | 12-15 주 | $\Delta = \text{acc}(\text{LogPress}) - \text{acc}(\text{baseline}) > 0$<br>AND paired bootstrap 95% CI 하한 > 0<br>AND McNemar $p < 0.05$ | floor 3단 백업 발동 (제약형 → DeepSeek → Tier-1 프레이밍)              |

#### 왜 사전 확정인가

임계값(AUC 0.65 등)은 관례적 선택이지 절대 기준이 아니다. 결과를 본 뒤 임계값을 조정하면 그 순간 검정은 무효가 된다. 팀 합의로 **실험 전에** 확정하고 논문에 근거를 명시하라.

## 1. 준비물 — 환경과 라이브러리 스택

Week 1 시작 전에 아래 스택을 고정한다. 버전 핀(pin)은 필수다.

```
pip install drain3 kvpress transformers accelerate torch ₩
scipy statsmodels scikit-learn pandas numpy ₩
matplotlib seaborn
```

| 목적                         | 라이브러리 / 모듈            | 핵심 함수                                                     |
|----------------------------|-----------------------|-----------------------------------------------------------|
| 로그 파싱(템플릿 추출)              | drain3                | TemplateMiner, add_log_message, match                     |
| KV 압축 & 모델                 | kvpress, transformers | ScorerPress, pipeline("kv-press-text-generation")         |
| 기술통계 · 비모수 검정              | scipy.stats           | mannwhitneyu, wilcoxon, permutation_test, bootstrap       |
| 짜진 검정 · 비율 CI · 검정력 · 다중보정 | statsmodels           | mcnemar, proportion_confint, TTestIndPower, multipletests |
| 분류 지표                      | sklearn.metrics       | roc_auc_score, roc_curve, accuracy_score                  |
| 로지스틱 회귀(2신호 결합)            | sklearn.linear_model  | LogisticRegression                                        |
| 데이터 처리                     | pandas, numpy         | DataFrame, groupby, np.percentile                         |
| 시각화                        | matplotlib, seaborn   | fill_between (CI 밴드), lineplot                            |

### 전 단계 공통 원칙 — 재현성

모든 난수(순열, 부트스트랩, split)에 **random\_state** / **seed**를 고정하고, split 인덱스를 파일로 저장하라. 재현되지 않는 실험은 결과가 아니다.

## 2. Weeks 1-3 — Kill-switch 실험 [관문 ①, D가 오너]

**목표:** "level 필터만으로 cause 라인이 얼마나 보존되는가"와 "필터가 놓친 cause를 rarity + position으로 건질 수 있는가"를 측정하여 프로젝트의 GO/NO-GO를 결정한다.

### 2.1 로그 파싱과 신호 계산 (drain3)

```
from drain3 import TemplateMiner
from drain3.template_miner_config import TemplateMinerConfig
import numpy as np, pandas as pd

config = TemplateMinerConfig()
config.load("drain3.ini") # sim_th=0.4, depth=4 기본
config.profilng_enabled = False
miner = TemplateMiner(config)

# logs: 한 인스턴트(컨텍스트)의 라인 리스트 [(level, message), ...]
def parse_context(logs):
    rows = []
    for i, (level, msg) in enumerate(logs):
        res = miner.add_log_message(msg) # 스트리밍 학습
        rows.append({"idx": i, "level": level,
                    "cluster_id": res["cluster_id"],
                    "template": res["template_mined"]})
    return pd.DataFrame(rows)
```

#### rarity(희귀도) 공식 — 템플릿 IDF

전체 코퍼스에서 템플릿 t가 df(t)개 라인에 등장하고 전체가 N 라인이면:

$$\text{rarity}(t) = \log(N / \text{df}(t)) \quad \# = \text{자기정보량 } \log(1/p), p = \text{df}/N$$

희귀한 템플릿일수록 rarity가 크다. "장애의 원인 라인은 대개 희귀하다"는 가정이 이 신호의 근거다.

```
def add_rarity(df):
    N = len(df)
    df_count = df.groupby("cluster_id")["idx"].transform("count")
    df["rarity"] = np.log(N / df_count)
    return df
```

#### position(위치) 공식 — 정규화 인덱스 + 중앙거리

$$\text{pos}(i) = i / (L-1) \quad \# \text{ 컨텍스트 길이 } L, \text{ pos} \in [0,1]$$
$$\text{mid\_dist}(i) = |\text{pos}(i) - 0.5| \quad \# 0\text{에 가까울수록 중앙} = \text{hard 영역}$$

```
def add_position(df):
    L = len(df)
    df["pos"] = df["idx"] / max(L - 1, 1)
    df["mid_dist"] = (df["pos"] - 0.5).abs()
    return df
```

### 2.2 Level 필터 보존율 측정 (기준선)

**ERROR|FAIL|EXCEPTION** 정규식으로 잡히는 라인만 남기는 단순 필터가 ground-truth cause 라인을 몇 % 살리는지 계산한다. 이 수치가 LogPress가 넘어야 할 기준선이다.

```
import re
LEVEL_RE = re.compile(r"(ERROR|FAIL|FATAL|EXCEPTION|CRITICAL)", re.I)

def level_filter_preservation(df, cause_idx):
    kept = df[df["template"].str.contains(LEVEL_RE) |
              df["level"].str.upper().isin(["ERROR", "FATAL", "CRITICAL"])]
    kept_idx = set(kept["idx"])
    preserved = sum(1 for c in cause_idx if c in kept_idx)
    return preserved / len(cause_idx) # 보존율
```

### 보조 판정식 (Auxiliary check)

전체 데이터에서 **level-filter** 보존율 < 0.99 여야 한다.

만약 0.99 이상이면 level 필터만으로 cause가 거의 다 잡힌다는 뜻 → LogPress의 rarity/position이 기여할 여지가 없다. **주제 재설정 신호**.

## 2.3 Hard case 정의와 표본 수 [GO 조건 ①]

**Hard case** = (a) non-ERROR(INFO/WARN) cause 라인 또는 (b) 컨텍스트의 중앙 25-75% 위치( $0.25 \leq pos \leq 0.75$ )에 있는 cause 라인. level 필터가 놓치기 쉬운, LogPress가 증명해야 할 표적이다.

```
def is_hard(row):
    non_error = row["level"].upper() not in ["ERROR", "FATAL", "CRITICAL"]
    middle = 0.25 <= row["pos"] <= 0.75
    return non_error or middle
```

### GO 판정식 ①

hard-case 총 표본 수  $n_{\text{hard}} \geq 300$  (수백 개 이상 = 통계적으로 유효한 최소 규모, \$2.5 검정력으로 정당화)

**NO-GO 대응:** ① BGL + Thunderbird 병합 ② stratification 완화 ③ 검정 단위를 인스턴트 → 라인 단위로 축소

## 2.4 신호 판별력 검정 [GO 조건 ②] — 핵심 통계

**질문:** "필터가 놓친 cause 라인"(hard positives)이 rarity/position 신호에서 "일반 noise 라인"과 유의하게 구분되는가?

**방법 A — 단일 신호: Mann-Whitney U (= AUC)**

rarity가 cause vs non-cause에서 분포가 다른지 검정한다. Mann-Whitney U 통계량은 AUC와 수학적으로 동치( $AUC = U / (n_1 \cdot n_2)$ )이므로 판별력을 직접 해석할 수 있다. **정규성 가정이 없어** 로그 데이터에 적합하다. 효과크기는 rank-biserial 상관( $r = 1 - 2U / (n_1 \cdot n_2)$ )으로 보고한다(|r| < .1 무시, .1-.3 소, .3-.5 중, > .5 대).

```
from scipy.stats import mannwhitneyu
from sklearn.metrics import roc_auc_score

def signal_discriminability(df, cause_idx):
    y = df["idx"].isin(cause_idx).astype(int).values    # 1 = cause
    x_rar = df["rarity"].values
    U, p = mannwhitneyu(x_rar[y==1], x_rar[y==0], alternative="greater")
    auc = roc_auc_score(y, x_rar)                    # = U / (n1*n2)
    r_rb = 1 - 2*U / ((y==1).sum() * (y==0).sum())    # rank-biserial 효과크기
    return auc, p, r_rb
```

**방법 B — 두 신호 결합: 로지스틱 회귀 + AUC**

rarity와 mid\_dist를 함께 넣어 결합 판별력을 본다. 단, **dev set에서만 학습**한다(라벨 누수 방지).

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

def combined_auc(df_dev, df_test, cause_dev, cause_test):
    Xtr = df_dev[["rarity", "mid_dist"]].values
    ytr = df_dev["idx"].isin(cause_dev).astype(int).values
    clf = LogisticRegression().fit(Xtr, ytr)    # dev에서만 fit
    Xte = df_test[["rarity", "mid_dist"]].values
    yte = df_test["idx"].isin(cause_test).astype(int).values
    return roc_auc_score(yte, clf.predict_proba(Xte)[:,1]) # test에서 평가
```

## 유의성 — 순열검정 (permutation test)

AUC가 우연(0.5)보다 유의하게 높은지, 라벨을 무작위로 섞어 만든 귀무분포와 비교한다. 분포 가정이 필요 없고 어떤 지표에도 적용 가능한 것이 장점이다.

```
import numpy as np

def auc_permutation_test(y, score, n_perm=5000, seed=0):
    rng = np.random.default_rng(seed)
    obs = roc_auc_score(y, score)
    null = np.empty(n_perm)
    for i in range(n_perm):
        null[i] = roc_auc_score(rng.permutation(y), score)
    p = (np.sum(null >= obs) + 1) / (n_perm + 1) # +1 보정
    return obs, p
```

### GO 판정식 ②

결합 신호  $AUC \geq 0.65$  AND 순열검정  $p < 0.05$

AUC 0.65는 "무작위(0.5)보다 확실히 낮고 실무적으로 의미 있는 하한"의 관례적 기준이다. 팀이 더 보수적으로 가려면 0.70으로 상향하되, 실험 전에 정하라.

**NO-GO 대응:**  $AUC \approx 0.5$ ,  $p \geq 0.05 \rightarrow$  신호 재설계(rarity를 엔트로피/시간 간격/버스트성으로 교체) 또는 주제 리셋.

## 2.5 검정력 분석 / 표본 수 계산 — "왜 300개인가"

GO 조건 ①의 숫자를 근거 있게 정당화한다. `statsmodels.stats.power.TTestIndPower`로 효과크기  $d = 0.5$ (중간),  $\alpha = 0.05$ ,  $power = 0.8$ 일 때 필요한 그룹당 표본을 구한다.

```
from statsmodels.stats.power import TTestIndPower

analysis = TTestIndPower()
n = analysis.solve_power(effect_size=0.5, power=0.8, alpha=0.05, ratio=1.0)
print(n) # ≈ 64 / 그룹
```

중간 효과( $d = 0.5$ )면 그룹당 약 64개로 충분하지만, hard-case는 불균형(cause가 소수)이고 McNemar/비율 검정은 t검정보다 표본이 더 필요하다. 따라서 안전계수를 두어  $n_{hard} \geq 300$ 을 목표로 삼는다.

이진 비율 비교의 검정력은 시뮬레이션이 더 정확하다:

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

es = proportion_effectsize(0.80, 0.65) # LogPress 0.80 vs baseline 0.65 가정
n_bin = NormalIndPower().solve_power(es, power=0.8, alpha=0.05, ratio=1.0)
print(n_bin)
```

## 3. Weeks 3-6 — 준비 (Stage 1)

### 3.1 보존율 스코어링 함수 사전 구현 (M5의 심장)

#### 핵심 매핑 문제

압축은 토큰 단위지만 cause는 라인 단위다. 따라서 (1) 토큰 → 라인 매핑, (2) 라인별 "보존 여부" 판정 규칙이 반드시 필요하다.

**보존 판정 규칙:** cause 라인의 토큰 중 **하나라도**(anyToken) KV가 살아있으면 "보존"으로 본다. RCA는 라인의 핵심 토큰 하나만 살아도 단서가 되기 때문이다. (대안: 과반(majority) 규칙)

```
import numpy as np

def build_line_token_map(line_texts, tokenizer):
    """각 라인이 차지하는 토큰 인덱스 구간 리스트 반환"""
    spans, cursor = [], 0
    for text in line_texts:
        n_tok = len(tokenizer(text, add_special_tokens=False)["input_ids"])
        spans.append(list(range(cursor, cursor + n_tok)))
        cursor += n_tok
    return spans          # spans[line_i] = [tok_idx, ...]

def preservation_rate(keep_mask, cause_line_idx, line_token_spans, rule="any"):
    """
    keep_mask   : np.bool_ [n_tokens] 압축 후 살아남은 토큰 True
    cause_line_idx : 정답 cause 라인 인덱스 리스트
    return      : (보존율 0~1, 라인별 보존 이진배열)
    """
    preserved = []
    for c in cause_line_idx:
        toks = line_token_spans[c]
        if len(toks) == 0:
            preserved.append(0); continue
        kept = keep_mask[toks]
        ok = kept.any() if rule == "any" else (kept.mean() >= 0.5)
        preserved.append(int(ok))
    preserved = np.array(preserved)
    return preserved.mean(), preserved
```

**keep\_mask**는 kvpress의 press가 남긴 인덱스로부터 구성한다 (§5.2 참조).

### 3.2 제약형 RCA 태스크 설계 (floor effect 회피)

Stage 5에서 "압축했더니 원래도 못 푸는 문제라 차이가 안 보이는" **바닥효과(floor effect)**를 피하려면 태스크를 쉽게 제약해야 한다.

- (A) **K지선다**: "다음 K개 후보 라인 중 근본원인은?" → 정답률(accuracy) 측정. **K = 4~5 권장**.
- (B) **라인 지목**: "원인 라인 번호를 출력하라" → top-1 정확도.

Nezha는 root-cause 라벨이 있어 K후보 구성이 가능하다(정답 1개 + distractor K-1개를 같은 인시던트에서 샘플링). Nezha는 Yu 외, ESEC/FSE 2023 (DOI 10.1145/3611643.3616249)에서 공개했고, OnlineBoutique·TrainTicket 두 마이크로서비스 시스템의 로그·메트릭·트레이스로 구성되며 코드 영역/리소스 유형 수준의 근본원인 라벨을 제공한다.

### 3.3 B의 Nezha 전처리 주간 점검

- root-cause 라벨 → 라인 인덱스 매핑 정합성 확인
- train/dev/test를 **인시던트** 단위로 분할했는지 확인 (라인 단위 분할은 누수!)
- 라벨 분포 통계 확보. 매주 15분 동기화.

## 4. Weeks 8-11 — 감독: 라벨 누수 방지 (Stage 3)

---

**임무:** LogPress의 가중치(level·rarity·time·position 결합 계수)를 **dev set**에서만 튜닝하는지 감시한다. 여기서 누수가 나면 Tier-1/Tier-2 결과 전체가 무효가 된다.

### 4.1 규율 (Discipline) 4원칙

- **3-way split**을 인시던트 단위로: train(신호 통계 · IDF 계산) / dev(가중치 튜닝) / test(최종 1회만). 전처리 (IDF의 N, df)는 **train**에서만 fit하고 dev/test에 적용(transform)한다.
- **test set**은 금고에 봉인: 모든 설계 결정이 끝난 뒤 딱 한 번 연다.
- **중첩 교차검증(nested CV)** 옵션: 표본이 작으면 외부 루프(성능평가) / 내부 루프(가중치 선택)로 분리해 튜닝 누수를 차단한다.
- **감사 로그:** 가중치 변경 커밋마다 "어느 split에서 나온 지표로 결정했는지" 기록한다.

```
from sklearn.model_selection import GroupShuffleSplit

# groups = 인시던트 ID → 같은 인시던트가 두 split에 걸치지 않게
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_dev_idx, test_idx = next(gss.split(X, y, groups=incident_ids))
```

## 5. Weeks 10-13 — Tier-1 평가 [M5, D가 주 오너 · "안전망"]

### 왜 안전망인가

보존율은 모델 정확도와 무관하게 측정된다. 따라서 Stage 5(RCA delta)가 바닥효과로 실패해도 Tier-1 결과만으로 논문이 선다. 여기를 반드시 완주하라.

### 5.1 LogPress를 kvpress 커스텀 press로 구현

kvpress의 `ScorerPress`를 상속해 `score()`만 구현하면 top-k 가지치기는 프레임워크가 처리한다. KV 텐서는 shape (batch, num\_kv\_heads, seq\_len, head\_dim)이며, `score()`는 shape (batch, num\_kv\_heads, seq\_len) 텐서를 반환하고 값이 클수록 보존된다. `compression_ratio`는 \*제거\* 비율이고 내부적으로 `n_kept = int(seq_len * (1 - compression_ratio))`로 상위 점수 토큰을 `torch.topk`로 남긴다.

#### 구현 주의

서브클래스는 반드시 `@dataclass` 여야 하며, 필드는 클래스 수준 annotation으로 선언한다 (`__init__` 정의 금지).

`score()` 반환 shape의 정답은 `keys.shape[:-1]` 이다. (내장 `KnormPress.score()`는 `return -keys.norm(dim=-1)` 한 줄이다.)

```
from dataclasses import dataclass
import torch
from kvpress.presses.scorer_press import ScorerPress

@dataclass
class LogPress(ScorerPress):
    compression_ratio: float = 0.0
    w_level: float = 1.0
    w_rarity: float = 1.0
    w_pos: float = 1.0
    # 사전 계산한 토큰별 로그 신호 (prefill 전에 주입)
    token_level: torch.Tensor = None # (seq_len,) ERROR=1 등
    token_rarity: torch.Tensor = None # (seq_len,) IDF
    token_mid: torch.Tensor = None # (seq_len,) |pos-0.5|

    def score(self, module, hidden_states, keys, values, attentions, kwargs):
        # keys: (bsz, num_kv_heads, seq_len, head_dim)
        bsz, n_heads, seq_len, _ = keys.shape
        s = (self.w_level * self.token_level
              + self.w_rarity * self.token_rarity
              - self.w_pos * self.token_mid) # 중앙(작은 mid)일수록 보존 ↑
        s = s.to(keys.device, keys.dtype) # (seq_len,)
        return s.view(1, 1, seq_len).expand(bsz, n_heads, seq_len)
```

사용 (컨텍스트 매니저):

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct").to("cuda:0")
press = LogPress(compression_ratio=0.7, token_level=lvl,
                 token_rarity=rar, token_mid=mid)

with torch.no_grad(), press(model):
    out = model(input_ids) # prefill 시 KV 압축 적용 (forward hook)
    kv = out.past_key_values # 압축된 캐시
```

`press(model)`은 각 `layer.self_attn`에 forward hook을 등록하고 블록을 벗어나면 자동 제거한다. 압축은 prefill 단계에서만 일어난다.

baseline은 kvpress 내장 press로: `RandomPress(Random)`, `StreamingLLMPress(StreamingLLM)`, `ObservedAttentionPress(H2O 유사)`, `PyramidKVPress(PyramidKV)`. 모두 동일 `compression_ratio`로 세팅해 공정 비교한다.

## 5.2 보존율 곡선 (압축비 50-90%)

```
import numpy as np, pandas as pd

ratios = [0.5, 0.6, 0.7, 0.8, 0.9]
methods = {"LogPress": mk_logpress, "Random": RandomPress,
           "StreamingLLM": StreamingLLMPress, "H2O": ObservedAttentionPress,
           "PyramidKV": PyramidKVPress}

records = []
for r in ratios:
    for name, mk in methods.items():
        press = mk(compression_ratio=r)
        for ctx in test_contexts:
            # held-out
            keep_mask = run_and_get_keepmask(model, press, ctx)
            for stratum, cidx in [("overall", ctx.cause),
                                ("nonERROR", ctx.cause_nonerror),
                                ("middle", ctx.cause_middle)]:
                pr, _ = preservation_rate(keep_mask, cidx, ctx.spans)
                records.append(dict(ratio=r, method=name, stratum=stratum,
                                   pres=pr, ctx_id=ctx.id))
df = pd.DataFrame(records)
```

### keep\_mask 추출 — 실무 함정

kvpress는 "남은 토큰 인덱스"를 공식 API로 노출하지 않는다. 가장 안전한 방법은 `score()` 로직을 그대로 재현해 상위 `n_kept = int(seq_len*(1-r))` 인덱스를 numpy로 직접 계산하는 것이다. 보존율 계산의 정확성이 여기 달려 있다.

## 5.3 필수 층화(Stratification) + 짝진 통계검정

3개 층: (a) overall (b) non-ERROR causes (c) middle-position causes. 각 층에서 LogPress vs 각 baseline을 McNemar 검정으로 비교한다. 라인별 "보존됨(1)/안됨(0)"이 두 방법에 대해 짝지어져 있기 때문이다.

```
from statsmodels.stats.contingency_tables import mcnemar
import numpy as np

def mcnemar_preservation(pres_A, pres_B):
    """pres_A, pres_B: 같은 cause 라인들에 대한 0/1 보존 배열 (짝)"""
    b = np.sum((pres_A==1) & (pres_B==0)) # A만 보존
    c = np.sum((pres_A==0) & (pres_B==1)) # B만 보존
    table = [[np.sum((pres_A==1)&(pres_B==1)), b],
             [c, np.sum((pres_A==0)&(pres_B==0))]]
    # 불일치쌍(b+c)이 적으면 exact=True 권장
    res = mcnemar(table, exact=(b+c < 25), correction=True)
    return res.statistic, res.pvalue, b, c
```

## 5.4 다중비교 보정 (BH FDR)

4 baseline × 3 strata × 5 압축비 = 최대 60개 p-value. Benjamini-Hochberg로 FDR을 보정하고, Holm으로 강건성을 점검한다. (Bonferroni는 Holm에 항상 열등하므로 단독으로 쓰지 마라.)

```
from statsmodels.stats.multitest import multipletests

pvals = df_tests["pvalue"].values
rej_bh, p_bh, _, _ = multipletests(pvals, alpha=0.05, method="fdr_bh")
rej_holm, p_holm, _, _ = multipletests(pvals, alpha=0.05, method="holm")

df_tests["p_bh"] = p_bh
df_tests["sig_bh"] = rej_bh
df_tests["sig_holm"] = rej_holm
```

### Tier-1 PASS 판정식

hard-case subset (non-ERROR + middle) 에서 LogPress가 4개 baseline 모두 대비 BH 보정 후  $p_{bh} < 0.05$ 로 유의한 보존율 우위를 보이고, 효과 방향이 일관될 것 ( $b > c$ , 즉 LogPress만 보존한 라인이 더 많음). overall에서만 이기는 건 불충분하다. overall은 level 필터도 잘하는 영역이기 때문이다.

## 5.5 Ablation 3중 설계

| Ablation | 비교 A                                         | 비교 B                       | 검정                    |
|----------|----------------------------------------------|----------------------------|-----------------------|
| ① 적용 시점  | early-layer 이후 적용 (FastKV 식 TSP, layer 15부터) | (a) layer 0부터 전층 (b) 층별 차등 | 층 설정별 보존율/정확도 McNemar |
| ② 필터 위치  | 입력단 필터링 (압축 전 라인 제거)                         | KV 압축 (LogPress)           | 동일 압축비에서 보존율 비교       |
| ③ 직렬 결합  | 필터 + KV 직렬 (ComposedPress)                   | KV 단독 (LogPress)           | 보존율 / 정확도 델타          |

```
from kvpress import ComposedPress, PerLayerCompressionPress

# ③ 직렬 결합 — ComposedPress는 press 리스트를 단일 위치인자로 받음
serial = ComposedPress([PreFilterPress(compression_ratio=0.3),
                        LogPress(compression_ratio=0.5, ...)])

# ① 층별 차등 (flash attention 필요; press 는 ScorerPress 여야 함)
per_layer = PerLayerCompressionPress(
    press=LogPress(...),
    compression_ratios=[0.0]*15 + [0.8]*17) # 초기 15층 보존, 이후 압축
```

**FastKV 근거** (Jo 외, arXiv:2502.01068, ACL Findings 2026): 초기 층은 전체 컨텍스트를 봐야 하고 후기 층은 소수 핵심 토큰에 집중하므로, Token-Selective Propagation(TSP) layer 이후에만 강하게 압축하면 정확도 손실이 작다. 논문은 32층짜리 **Llama-3.1-8B-Instruct**에서 **layer 15**를 TSP layer로 선택했다고 명시한다.

## 6. Weeks 12-15 — Tier-2 평가: RCA 델타 [M6, "결정타"]

### 6.1 실험 설계: 압축비 고정, 방법만 교체

동일 압축비에서 압축 \* 방법만 바뀌 Llama-3.1-8B에 넣고 제약형 RCA 정확도의 델타( $\Delta = \text{acc\_LogPress} - \text{acc\_baseline}$ )를 측정한다. 같은 인시던트 집합이므로 짝진 설계다.

```
from transformers import pipeline

pipe = pipeline("kv-press-text-generation",
                model="meta-llama/Llama-3.1-8B-Instruct",
                device="cuda:0",
                model_kwargs={"attn_implementation": "flash_attention_2"})

def rca_accuracy(press, eval_items):
    correct = []
    for it in eval_items:  # it.context, it.question(K지선다), it.answer
        ans = pipe(it.context, question=it.question, press=press)["answer"]
        correct.append(int(parse_choice(ans) == it.answer))
    return np.array(correct)  # 인시던트별 0/1
```

파이프라인은 context만 압축하고 question은 압축하지 않는다.

### 6.2 정확도 델타 + 유의성 + 신뢰구간

```
from statsmodels.stats.contingency_tables import mcnemar
from statsmodels.stats.proportion import proportion_confint
from scipy.stats import bootstrap
import numpy as np

acc_log = rca_accuracy(logpress, eval)  # 0/1 배열
acc_base = rca_accuracy(pyramidkv, eval)

# (1) 짝진 유의성: McNemar
b = np.sum((acc_log==1) & (acc_base==0))
c = np.sum((acc_log==0) & (acc_base==1))
res = mcnemar([np.sum((acc_log==1)&(acc_base==1)), b],
              [c, np.sum((acc_log==0)&(acc_base==0))],
              exact=(b+c < 25))

# (2) 델타의 부트스트랩 95% CI (짝진 차이)
diff = acc_log - acc_base
ci = bootstrap((diff,), np.mean, confidence_level=0.95,
               n_resamples=10000, method="BCa", random_state=0)

# (3) 각 방법 정확도의 Wilson CI
lo_l, hi_l = proportion_confint(acc_log.sum(), len(acc_log), method="wilson")

print(res.pvalue, ci.confidence_interval, (lo_l, hi_l))
```

#### Tier-2 PASS 판정식

$\Delta > 0$  AND paired bootstrap 95% CI 하한  $> 0$  (CI가 0을 포함하지 않음) AND McNemar  $p < 0.05$

Wilson 구간은 개별 정확도 보고용이다. 표본이 작거나 정확도가 0/1에 가까울 때 정규근사(Wald)는 구간이 [0,1]을 벗어나므로 Wilson score 구간이 표준이다.

### 6.3 2번째 백본 재현 (일반화)

Qwen2.5-7B-Instruct로 동일 실험을 반복한다. 두 백본에서 방향이 일치하면 일반화 주장이 강해진다.

6.4 FLOOR BACKUP 3단계 (바닥효과 대비)

| 단계 | 대응                                                                          | 트리거                               |
|----|-----------------------------------------------------------------------------|-----------------------------------|
| 1차 | 제약형 태스크(K지선다)로 Llama 바닥효과 회피                                                | 기본 설계                             |
| 2차 | 추론 특화 DeepSeek-R1-Distill-Qwen-7B로 교체                                       | 1차에서도 정확도가 바닥(모든 방법 $\approx 0$ ) |
| 3차 | Stage 5를 "방향성 관찰(directional observation)"로 강등하고, Stage 4(보존율)를 핵심 결과로 프레이밍 | 2차에서도 델타 측정 불가                    |

보존율은 모델 정확도와 독립적으로 측정되므로 **바닥효과에 강건**하다. 이것이 Tier-1을 안전망이라 부르는 이유다.

## 7. Weeks 13-16 — 마무리 [M7, D 총괄]

### 7.1 Novelty 방어 (핵심 프레이밍)

"단순 level 필터 대비 rarity/position 신호의 한계 이득(marginal gain)"을 핵심 결과로 세운다.

```
marginal_gain = 보존율(LogPress) - 보존율(level-filter-only)
```

hard-case에서 이 이득이 BH 보정 후에도 유의하면, "level 필터로 충분하지 않은가?"라는 리뷰어 반박을 정면 방어할 수 있다. Kill-switch(§2)의 AUC 결과가 이 서사의 복선이다.

### 7.2 실패 사례 분석

LogPress가 놓친 cause 라인의 공통점을 분석한다(예: 매우 흔한 템플릿이면서 중앙에 있는 경우). `df`에서 `preserved == 0`인 라인의 rarity/pos 분포를 시각화한다.

### 7.3 시각화 — 압축비 vs 보존율 곡선 + 부트스트랩 CI 밴드

```
import matplotlib.pyplot as plt
import numpy as np

def bootstrap_band(vals, n=2000, seed=0):
    rng = np.random.default_rng(seed)
    means = [rng.choice(vals, len(vals), replace=True).mean() for _ in range(n)]
    return np.mean(vals), np.percentile(means, 2.5), np.percentile(means, 97.5)

fig, ax = plt.subplots(figsize=(7,5))
for name, g in df[df.stratum=="middle"].groupby("method"):
    xs, ms, los, his = [], [], [], []
    for r, gr in g.groupby("ratio"):
        m, lo, hi = bootstrap_band(gr["pres"].values)
        xs.append(r); ms.append(m); los.append(lo); his.append(hi)
    ax.plot(xs, ms, marker="o", label=name)
    ax.fill_between(xs, los, his, alpha=0.2) # CI 밴드

ax.set_xlabel("Compression ratio")
ax.set_ylabel("Preservation rate (middle causes)")
ax.legend(); plt.tight_layout()
plt.savefig("preservation_curve.png", dpi=300)
```

seaborn을 쓰면 `sns.lineplot(data=df, x="ratio", y="pres", hue="method", errorbar=("ci", 95))`로 CI 밴드가 자동 생성된다.

## 8. 통계 선택의 근거 — "왜 이 검정인가"

### 8.1 왜 McNemar인가 (짜진 이진 결과)

LogPress와 baseline은 동일한 cause 라인 집합에 적용되므로 관측치가 짝지어져 있다. 이진 결과(cause 라인의 KV가 보존됐나 아닌가)에는 McNemar 검정이 표준이다.

#### 고전 근거

Dietterich, "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms," \*Neural Computation\* 1998;10(7):1895-1923 은 한 번만 학습·실행하는 알고리즘 비교에서는 McNemar 검정만이 허용 가능한 제1종 오류율을 갖는다고 결론짓는다.

즉 8B 모델을 (재학습 없이) 한 번 돌려 held-out에서 비교하는 우리 상황에 정확히 맞는 검정이다. 독립표본 검정(카이제곱, 독립 t-검정)은 짝 구조를 무시해 검정력을 낭비하므로 틀렸다.

연속 지표(정확도 델타의 표본별 분포)나 분포 가정을 피하고 싶을 때는 paired bootstrap 또는 Wilcoxon signed-rank를 쓴다.

### 8.2 왜 AUC + 순열검정인가 (Kill-switch)

"rarity/position 신호가 cause 라인을 noise와 구분하는가"는 본질적으로 이진 분류의 판별력 문제다. 단일 신호는 Mann-Whitney U(= AUC와 동치), 두 신호 결합은 로지스틱 회귀 + ROC-AUC로 측정하고, 유의성은 라벨을 섞는 순열검정으로 확인한다. 정규성 가정이 필요 없고 임의 지표에 적용 가능한 것이 장점이다.

### 8.3 왜 Benjamini-Hochberg인가 (다중비교)

4 baseline × 3 strata × 여러 압축비 = 수십 개 검정이므로 보정이 필수다. 최근 ML/NLP 논문의 관행은 BH(FDR)를 주 분석으로, Holm-Bonferroni(FWER)를 보수적 점검으로 병기하는 것이다.

### 8.4 임계값을 바꿔야 하는 신호

- AUC가 0.55-0.65 애매구간 → 순열검정 p와 효과크기(rank-biserial)를 함께 보고, 신호 추가(시간 간격, 버스트성) 검토.
- 불일치쌍 (b+c) < 25 → McNemar exact=True(이항검정)로 전환.
- 표본이 매우 작음 (< 100) → t검정 대신 Wilcoxon signed-rank + 시뮬레이션 기반 검정력.

## 9. 주의사항 (Caveats)

### 9.1 통계 · 설계

- AUC 0.65 임계값은 관례적 선택이지 절대 기준이 아니다. 팀 합의로 사전 확정하고 논문에 근거를 명시하라 (사후 조정은 p-hacking).
- McNemar는 불일치쌍이 충분해야 검정력이 있다. 두 방법이 거의 같은 라인을 보존/누락하면  $b+c$ 가 작아 검정력이 급감한다 → 압축비를 높여 차이를 키우거나 exact 검정 사용.

### 9.2 도구 · 라이브러리

- `kvpress` ↔ `transformers` 버전 호환성 주의. `kvpress`는 `transformers`의 KV 캐시 구조 변경에 맞춰 breaking change를 겪은 이력이 있다. `ScorerPress` / `compress()` 시그니처는 설치한 버전의 소스 (`kvpress/presses/scorer_press.py`, `base_press.py`)로 최종 확인하라.
- `ObservedAttentionPress`는 H2O와 "유사"할 뿐 동일하지 않다 (원논문과 세부 상이). 또한 `attn_implementation="eager"`가 필요하다 (flash attention 비호환). `PyramidKV` · `StreamingLLM`은 flash attention 호환.
- `PerLayerCompressionPress`는 실험적(experimental) 기능이며 flash attention에서만 동작한다고 `kvpress`가 명시한다. Ablation ①에서 오류 시 대안 경로(전층 균일 압축 vs layer-0 압축 비교)를 준비하라.
- `keep_mask` 추출은 `kvpress`가 공식 API로 노출하지 않는다. `score()`를 재현해 top-k 인덱스를 직접 계산하는 방식이 가장 안전하다.

### 9.3 모델 · 데이터

- FastKV의 TSP layer = 15는 Llama-3.1-8B(32층) 기준이다. Qwen2.5-7B / DeepSeek는 층 수·구조가 달라 재탐색이 필요하다.
- BGL / Thunderbird는 anomaly(alert/non-alert) 라벨만 있고 root-cause 라벨이 없다. 따라서 Tier-2 RCA 정확도는 Nezha에서만 가능하고, BGL/Thunderbird는 보존율(Tier-1) 중심으로 쓴다.
- Nezha는 상대적으로 저부하(low-QPS) 벤치마크로, 실제 고동시성 장애를 완전히 대표하지는 않는다. 일반화 주장 시 한계로 명시하라.

#### 한 문장 요약

Kill-switch를 AUC + 순열검정으로 통과시키고(3주), 보존율을 McNemar + BH 보정으로 방어해 안전망을 확보한 뒤(13주), RCA 델타의 부트스트랩 CI 하한이 0을 넘으면 결정타를 날린다(15주). 모든 판정식은 실험 전에 확정한다.